深度強化學習 HW1

網格地圖開發 & 策略評估

Grid Map Development & Policy Evaluation

開發對話記錄

日期：2026-03-12

GitHub: https://github.com/albertiscool/hw1

目錄

1. 環境安裝 — 安裝 Git
2. HW1-1 & HW1-2 實作 — 網格地圖 + 策略評估
3. 推送至 GitHub
4. 策略評估分析 — 對照作業圖片
5. HW1-3 實作 — 價值迭代 + 策略萃取 + 簡化 UI
6. 修正策略評估 — 確定性 → 隨機策略
7. README 與文件產生

1. 環境安裝 — 安裝 Git

USER

幫我安裝 git

ASSISTANT

檢查系統：git 尚未安裝。

使用 winget（Windows 套件管理員）安裝：

```
> winget install --id Git.Git
```

安裝完成！驗證：git version 2.53.0.windows.2

提醒使用者需重啟終端機才能直接使用 git 指令。

2. HW1-1 & HW1-2 實作

USER

幫我實作
（附上作業截圖：HW1-1 網格地圖開發 60% + HW1-2 策略評估 40%）

ASSISTANT

分析作業需求後完成實作：

[app.py — Flask 後端]
- 路由 /evaluate：接收網格設定，隨機生成策略並執行 Policy Evaluation
- Bellman 方程式：V(s) = R + gamma * V(s')
- 參數：gamma=0.9, reward_step=-1, reward_goal=10

[index.html — 前端介面]
- 輸入 n (5~10) 生成 n x n 網格
- 點擊設定起點(綠)、終點(紅)、障礙物(灰)
- 障礙物支援 toggle 切換
- 結果顯示 Value Matrix 和 Policy Matrix

啟動 Flask 伺服器並在瀏覽器測試，全部功能正常。

3. 推送至 GitHub

USER

幫我 push 到我的 github
https://github.com/albertiscool/hw1.git

ASSISTANT

操作步驟：
1. git init — 初始化倉庫
2. 建立 .gitignore（排除 __pycache__、.env 等）
3. git config 設定使用者資訊
4. git add -A -> git commit -> git push origin main

推送成功！3 個檔案已上傳。

4. 策略評估分析 — 對照作業圖片

USER

目前的策略評估函數用的是甚麼，有用到這張圖上講的嗎？
（附上「網格世界強化學習實作指南」圖片）

ASSISTANT

分析結果：

[已使用 — Bellman 方程式]
V(s) = R(s,a) + gamma * V(s')
對應圖片「階段二」的核心運算

[圖上的演算法比較表]
- 策略評估 (PE)：動作機率加權和 -> 估算給定策略的價值
- 價值迭代 (VI)：取最大動作價值 (Max) -> 找最佳價值函數
- 策略萃取：Argmax 運算 -> 決定最佳動作

[尚未實作 — HW1-3 階段三]
- 價值迭代 (Value Iteration) 用 Max
- 策略萃取 (Policy Extraction) 用 Argmax

5. HW1-3 實作 — 價值迭代 + 簡化 UI

USER

幫我實作，然後順便幫我把整體風格用的陽春一點，像是這張圖就好
（附上作業截圖，白色背景的簡單風格）

ASSISTANT

[A. UI 風格簡化]
從深色 glassmorphism 改為白色背景簡潔風格
符合作業截圖的樸素排版

[B. 新增 HW1-3 功能]
後端新增 /value_iteration 路由
價值迭代：V(s) = max_a [R(s,a) + gamma * V(s')]
策略萃取：pi*(s) = argmax_a [R(s,a) + gamma * V*(s')]
最佳路徑萃取並用黃色標記

[結果比較]
PE 迭代 ~133 次，VI 只需 ~9 次就收斂

5x5 網格設定：起點(S)、終點(E)、障礙物

6. 修正策略評估 — 隨機策略加權和

USER

為甚麼我的隨機策略不像我給的圖那樣看起來不一樣，而是全都是 -10

ASSISTANT

[問題原因]
之前用「確定性隨機策略」每格只隨機選一個方向。
大部分路徑形成循環，永遠到不了終點。
值收斂到 -1/(1-0.9) = -10

[正確做法 — 隨機策略 (Stochastic Policy)]
V(s) = Sum pi(a|s) * [R(s,a) + gamma * V(s')]
= 0.25*[R+gV(上)] + 0.25*[R+gV(下)] + 0.25*[R+gV(左)] + 0.25*[R+gV(右)]

[修正結果]
每個格子有不同的值：
- 起點附近 = -9.71
- 終點旁邊 = -0.17
- 靠近終點的格子價值較高

核心差異：
X 確定性：每格只看 1 個動作 -> 循環 -> 全 -10
O 隨機策略：4 個動作各 25% 加權和 -> 部分機率到終點 -> 不同值

修正後的 Policy Evaluation：各格子有不同值

7. 最終成果與 README

HW1-2 策略評估 (上) vs HW1-3 價值迭代 (下)

USER

幫我產生 readme.md 然後上傳 github，readme 裡要有程式 demo

ASSISTANT

建立 README.md 包含：

- 作業說明（HW1-1, HW1-2, HW1-3）
- 技術架構（Flask / HTML / AJAX）
- 安裝與執行指令
- 核心演算法公式
- 3 張 Demo 截圖
- 檔案結構與演算法比較表

已推送至 https://github.com/albertiscool/hw1

演算法比較：

策略評估 (PE) | 加權和 (Sum) | ~120+ 迭代

價值迭代 (VI) | 最大值 (Max) | ~9 迭代

策略萃取 | Argmax | 1 次遍歷